# CyberTriage

A rule-based expert system for triaging cyber incidents

Code explanation | Introduction to AI, Complex Computing Problem


| | |
|---|---|
| **Name** | Ahmed Raza |
| **Roll No.** | CT-24171 |
| **Course** | Introduction to AI |
| **Live app** | https://cybertriage.streamlit.app/ |
| **Source code** | https://github.com/ahmed2402/Expert-System-IAI- |
| **Submitted** | November 2026 |


67 rules | 51 questions | 15 attack profiles | forward chaining with certainty factors | Streamlit UI

# 1. What CyberTriage does

Most people who get hacked don't know they've been hacked. They notice odd things: files that won't open, a phone that suddenly shows 'No service', a laptop fan screaming while nothing is running. Then they either panic or ignore it. CyberTriage is meant for that moment. You tell it what you've noticed, and it works out which attack is most likely behind it, how serious it is, and what you should do first.

I picked this niche because most expert-system projects I've seen are medical or plant-disease diagnosis, and cyber incidents fit the expert-system model surprisingly well. Security people triage by rules of thumb: 'files renamed to .locked plus a ransom note means ransomware', 'lost signal right after a SIM change message means SIM swap'. Those rules of thumb are exactly what an expert system stores. I also made the advice local where it matters, so it mentions JazzCash and Easypaisa, the PTA, and the NCCIA (the agency that took over from the FIA Cyber Crime Wing).

The system has three parts:

- an inference engine I wrote from scratch, with no expert-system library, that does forward chaining and handles uncertainty with MYCIN-style certainty factors
- a knowledge base of 67 rules, 51 questions and 15 attack profiles, each with its own response steps
- a Streamlit web app that asks the questions, shows the diagnosis, explains the reasoning and produces a PDF incident report

The app is live at https://cybertriage.streamlit.app/ and anyone can open it.

# 2. Using the app

I kept the app to three steps, because an earlier version that showed all 51 questions on one screen felt like filling in a tax form.

### Step 1: where's the trouble?

You tap the areas where something seems wrong, like Files or Phone & SIM, and set four switches about your situation (work device, money involved, sensitive data, other devices affected). The switches don't change which attack it finds, but they change how severe it is and which urgent actions it adds.

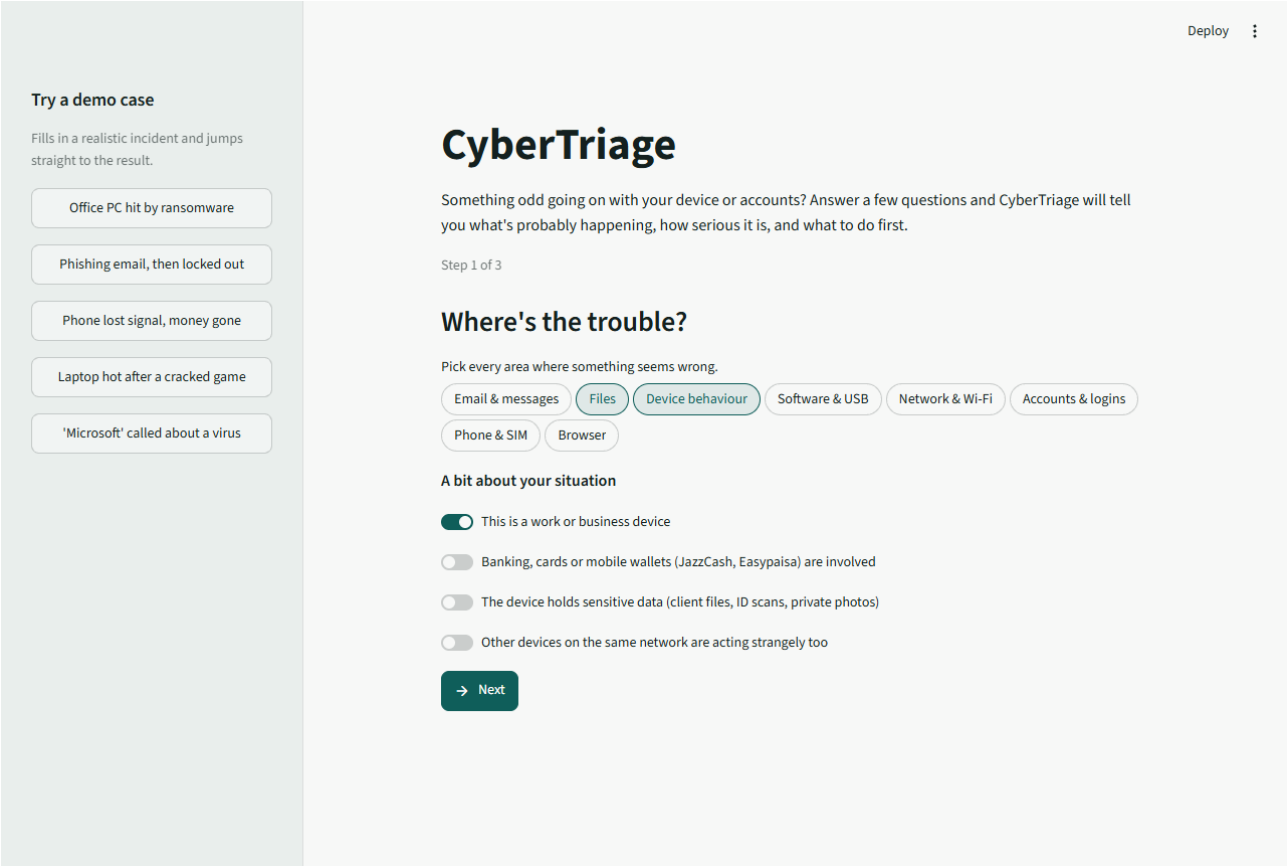



*Figure 1. Step 1: picking the problem areas and describing the situation.*

## Step 2: what have you noticed?

Only the questions for the areas you picked appear. Each one can be left blank, answered Yes, or answered Not sure. Yes counts as certainty 1.0 and Not sure as 0.5, so a half-remembered symptom still counts, just with less weight.

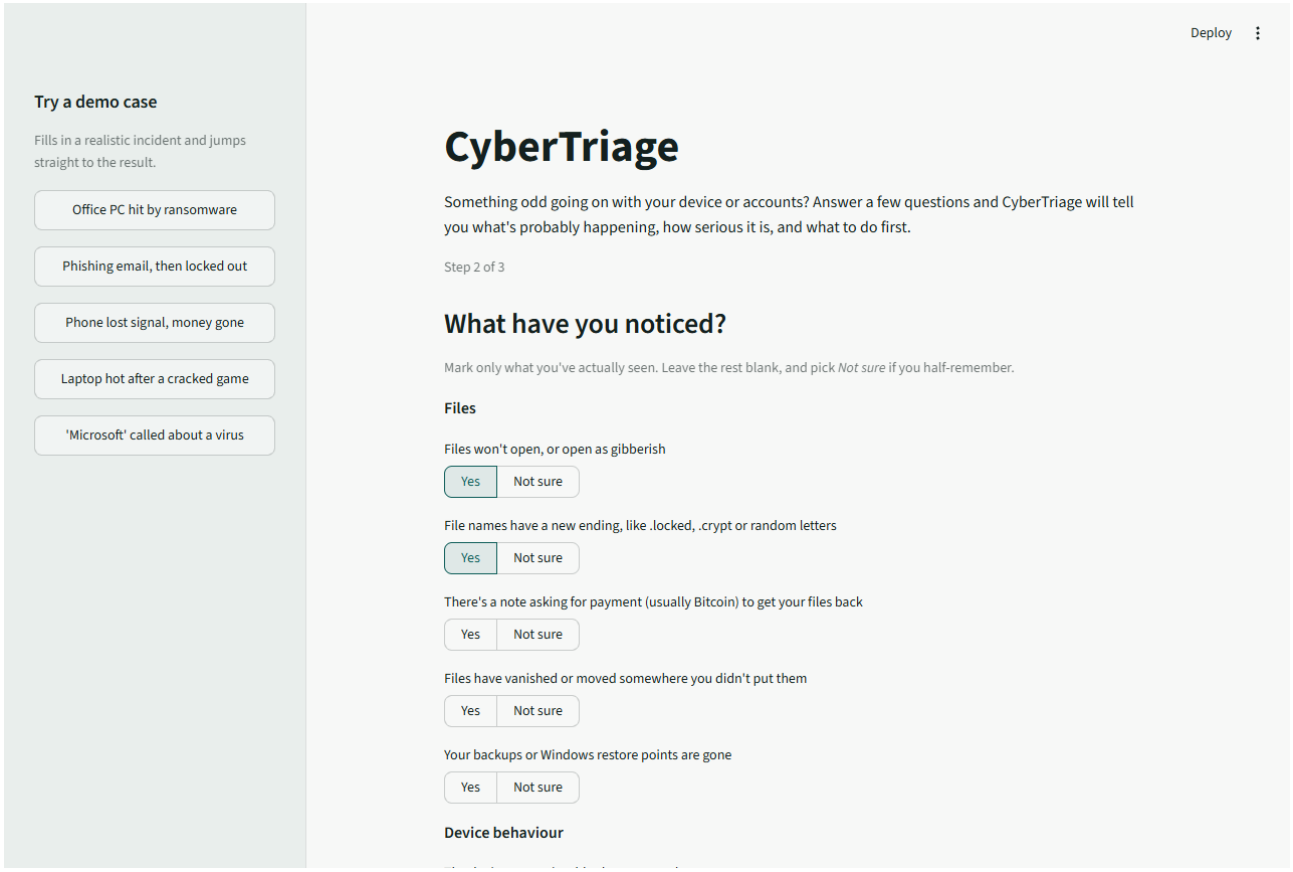



*Figure 2. Step 2: only the questions for the chosen areas.*

## Step 3: the diagnosis

The result page opens with a coloured severity banner, then a short 'do these first' list, then each likely attack with its confidence, a plain description and numbered response steps. From here you can download the whole thing as a PDF report, start over, or go back and change answers. The sidebar has five demo cases that fill everything in and jump straight here, which is handy for a quick demo.

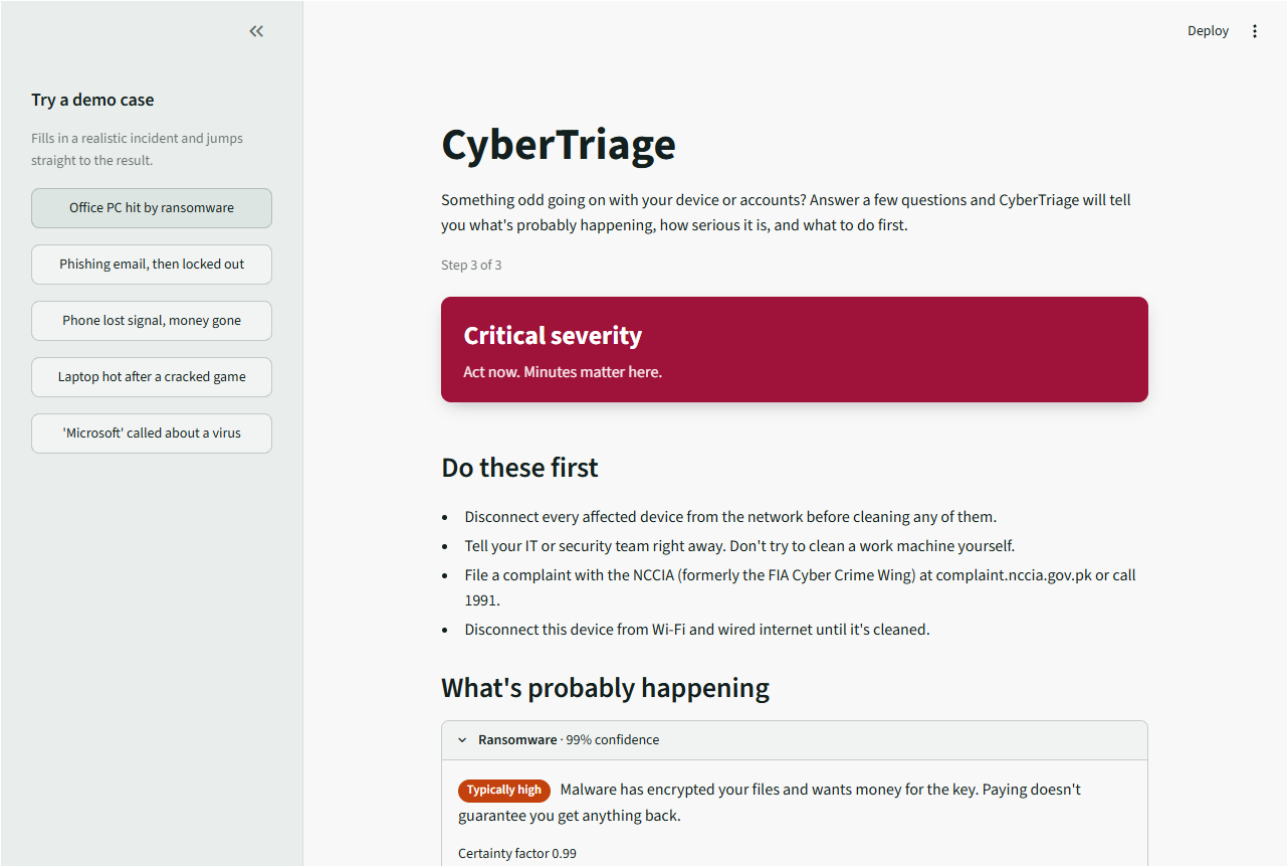



*Figure 3. The result for the 'Office PC hit by ransomware' demo case.*

## 3. How the code is organised

The project is four Python files. I kept the rules as data rather than writing them as if/else statements, so the engine never has to change when a rule is added, and the rule count is easy to check.

| File | What it does |
|---|---|
| engine.py | The inference engine: the Rule class, forward chaining, certainty factor maths, and the 'why not?' check. About 70 lines. |
| knowledge_base.py | Everything the system knows: the questions, the 67 rules, the 15 attack profiles, the extra actions, and the five demo cases. Also diagnose(), which runs the engine and ranks the results. |
| app.py | The Streamlit interface: the three-step wizard, the result page, the explanation panels and the PDF report. |
| test_engine.py | Plain assert checks for the engine and the knowledge base, including one check per demo case. |
| requirements.txt | streamlit and fpdf2, the only two packages the app needs. |

# 4. The inference engine (engine.py)

## 4.1 What a rule looks like

Each rule is a small dataclass. if_all holds the facts that must all be true (AND). if_any holds facts where at least one must be true (OR); it's optional. then is the new fact the rule concludes, and cf is how much I trust the rule, from 0 to 1. because is a plain-English reason that the app shows the user.

```
@dataclass
class Rule:
    id: str
    if_all: list          # every one of these facts must be known
    then: str             # fact this rule concludes
    cf: float             # how much the expert trusts this rule (0..1)
    if_any: list = field(default_factory=list)  # plus at least one of these, if given
    because: str = ""     # plain-language reason, shown in the explanation tab
```

Here is one real rule from the knowledge base. It says that unreadable files with a new extension point to encryption, and I trust that rule at 0.90:

```
R("R01", ["files_unreadable", "odd_extensions"], "files_encrypted", 0.90,
  because="Unreadable files with a new extension are the classic sign of encryption.")
```

## 4.2 Certainty factors

Expert systems rarely deal in pure true or false, and cyber incidents definitely don't. Strange CPU use could be a crypto-miner, or it could be Windows Update. I used the certainty-factor model from MYCIN, the classic medical expert system from Stanford. Every fact carries a number between 0 and 1, and three small rules decide how the numbers flow:

- AND takes the weakest premise. A chain is only as strong as its weakest link, so a rule needing two facts at 1.0 and 0.5 starts from 0.5.
- OR takes the strongest premise. One good piece of evidence is enough.
- The result is the premise strength multiplied by the rule's own CF.

The fourth rule is the interesting one. When two separate rules reach the same conclusion, the evidence should add up, but it must never go past 1. MYCIN's combination formula does that:

```
def combine(old, new):
    """MYCIN combination for two positive pieces of evidence for the same fact."""
    return old + new * (1 - old)
```

If one rule says 0.6 and another says 0.5, the result is 0.6 + 0.5 x 0.4 = 0.8. Each new piece of evidence closes part of the remaining gap, which matches how confidence really grows.

```
def premise_cf(rule, facts):
    """Certainty of a rule's IF part, or None if the rule can't fire."""
    if any(facts.get(f, 0) <= 0 for f in rule.if_all):
        return None
    cfs = [facts[f] for f in rule.if_all]
    if rule.if_any:
        known = [facts[f] for f in rule.if_any if facts.get(f, 0) > 0]
        if not known:
            return None
        cfs.append(max(known))  # OR = strongest supporting fact
    return min(cfs)              # AND = weakest link
```

## 4.3 Forward chaining

infer() is the heart of the engine. It starts from the facts the user gave and keeps passing over the rule list. Whenever a rule's conditions are met, it fires, its conclusion becomes a new fact, and that fact can trigger other rules on the next pass. The loop stops when a full pass fires nothing new. Each rule fires once at most, so the loop always ends. Every firing goes into the trace, which is what the app later shows as the explanation.

```
def infer(facts, rules):
    """Forward chaining. Returns (all facts, trace of (rule, cf) in firing order)."""
    facts = dict(facts)
    fired, trace = set(), []
    changed = True
    while changed:                       # repeat passes until nothing new fires
        changed = False
        for rule in rules:
            if rule.id in fired:
                continue
            p = premise_cf(rule, facts)
            if p is None:
                continue
            cf = round(p * rule.cf, 3)
            facts[rule.then] = round(combine(facts.get(rule.then, 0), cf), 3)
            fired.add(rule.id)
            trace.append((rule, cf))
            changed = True
    return facts, trace
```

I chose forward chaining over backward chaining because the user starts with symptoms, not with a guess. Data-driven reasoning fits that better. It also means the system can report several attacks at once, which happens a lot: a phishing email often leads to an account takeover.

## 4.4 A worked example

Say the user marks four things as Yes: files won't open, the file names changed, there's a ransom note, and the backups are gone. Here's what the engine does, using the real numbers it produces:

- R01 fires with CF 0.900, so 'files_encrypted' = 0.900.
- R02 fires with CF 0.800. 'files_encrypted' already had 0.900, so they combine: 0.900 + 0.800 x (1 - 0.900) = 0.980.
- R22 fires with CF 0.882, so 'attack_ransomware' = 0.882.
- R23 fires with CF 0.931. 'attack_ransomware' already had 0.882, so they combine: 0.882 + 0.931 x (1 - 0.882) = 0.992.
- R66 fires with CF 0.794, so 'act_report_nccia' = 0.794.
- R67 fires with CF 0.893, so 'act_disconnect' = 0.893.

So ransomware comes out at 99% confidence. Notice the chaining: R22 and R23 could only fire because R01 and R02 had already concluded 'files_encrypted'. Nobody typed that fact in. The engine worked it out.

### 4.5 Explaining a 'no'

A good expert system should also explain why it did not reach a conclusion. why_not() looks at every rule that could have concluded a given attack and lists the conditions that were missing. The app uses this for its 'Why not something else?' panel.

```
def why_not(goal, facts, rules):
    """For each rule that could conclude `goal`, list the premises that are missing."""
    out = []
    for r in rules:
        if r.then != goal:
            continue
        missing = [f for f in r.if_all if facts.get(f, 0) <= 0]
        if r.if_any and not any(facts.get(f, 0) > 0 for f in r.if_any):
            missing.append("one of: " + ", ".join(r.if_any))
        out.append((r, missing))
    return out
```

## 5. The knowledge base (knowledge_base.py)

The knowledge base has 67 rules arranged in three layers. Splitting them this way keeps each rule small and makes the chaining visible.

- Layer 1 (21 rules) turns raw symptoms into warning signs, for example 'CPU busy while idle' plus 'loud fan' becomes 'something is secretly using the hardware'.
- Layer 2 (31 rules) turns warning signs into attacks. Most attacks have two or three rules pointing at them, so stronger evidence gives a higher combined confidence.
- Layer 3 (15 rules) looks at the attack plus the situation. Ransomware on a work machine is raised to Critical, and anything involving a bank account adds 'call your bank now'.

The 15 attacks it can recognise are: Ransomware, Phishing, Account takeover, Spyware / keylogger, Cryptojacking, Botnet infection, Adware / browser hijacker, Trojan / fake app, SIM swap, Social media hijack, Tech-support scam, Man-in-the-middle / rogue Wi-Fi, USB-borne malware, Data breach exposure, Business email compromise.

Each attack profile stores a name, a typical severity, a one-line description in plain language, and four or five response steps in the order you should do them:

```
"attack_sim_swap": ("SIM swap", "Critical",
    "Someone convinced your telco to move your number to their SIM, so your OTPs now go to them.",
    ["Call your telco from another phone or visit a franchise with your CNIC. ...",
     "Call your bank and wallet helplines to freeze accounts.", ...]),
```

### 5.1 diagnose(): from facts to an answer

diagnose() is the bridge between the engine and the app. It runs infer(), keeps every attack whose confidence is at least 0.30 (anything lower is too weak to be worth showing), sorts them, and works out the overall severity. The severity starts at the worst typical severity among the attacks found, and layer 3 rules can push it up to High or Critical.

```
def diagnose(answers):
    """answers: {fact_id: cf}. Returns (ranked attacks, overall severity, actions, facts, trace)."""
    from engine import infer
    facts, trace = infer(answers, RULES)
    attacks = sorted(((a, facts[a]) for a in ATTACKS if facts.get(a, 0) >= DIAGNOSIS_THRESHOLD),
                     key=lambda x: -x[1])
    level = max((SEVERITY_ORDER.index(ATTACKS[a][1]) for a, _ in attacks), default=-1)
    if facts.get("sev_high", 0) >= 0.5:
        level = max(level, 2)
    if facts.get("sev_critical", 0) >= 0.5:
        level = 3
    severity = SEVERITY_ORDER[level] if level >= 0 else None
    actions = [ACTIONS[a] for a in ACTIONS if facts.get(a, 0) >= 0.5]
    return attacks, severity, actions, facts, trace
```

# 6. The Streamlit app (app.py)

## 6.1 Keeping answers between steps

Streamlit reruns the whole script every time you click something, and it throws away the state of any widget that isn't on screen. That was a problem for a wizard: answers given in step 2 would vanish by step 3. I fixed it by storing the answers in my own session-state dictionary and copying each change into it through an on_change callback. The widgets only display what's stored.

```
ss.setdefault("ans", {})      # fact_id -> "Yes" / "Not sure"

def save_answer(fid):
    value = ss["w_" + fid]
    if value:
        ss.ans[fid] = value
    else:
        ss.ans.pop(fid, None)
```

When the result page needs the facts, current_answers() turns 'Yes' into 1.0 and 'Not sure' into 0.5, adds the situation switches, and hands the lot to diagnose().

## 6.2 The explanation facility

Under the result there's a section called 'Under the hood' with three panels. The first lists every rule that fired, in the order it fired, written out as IF ... THEN ... with its certainty factor and its plain reason. The second is the 'why not?' check. The third shows all 67 rules in a table, so anyone can see the whole knowledge base.

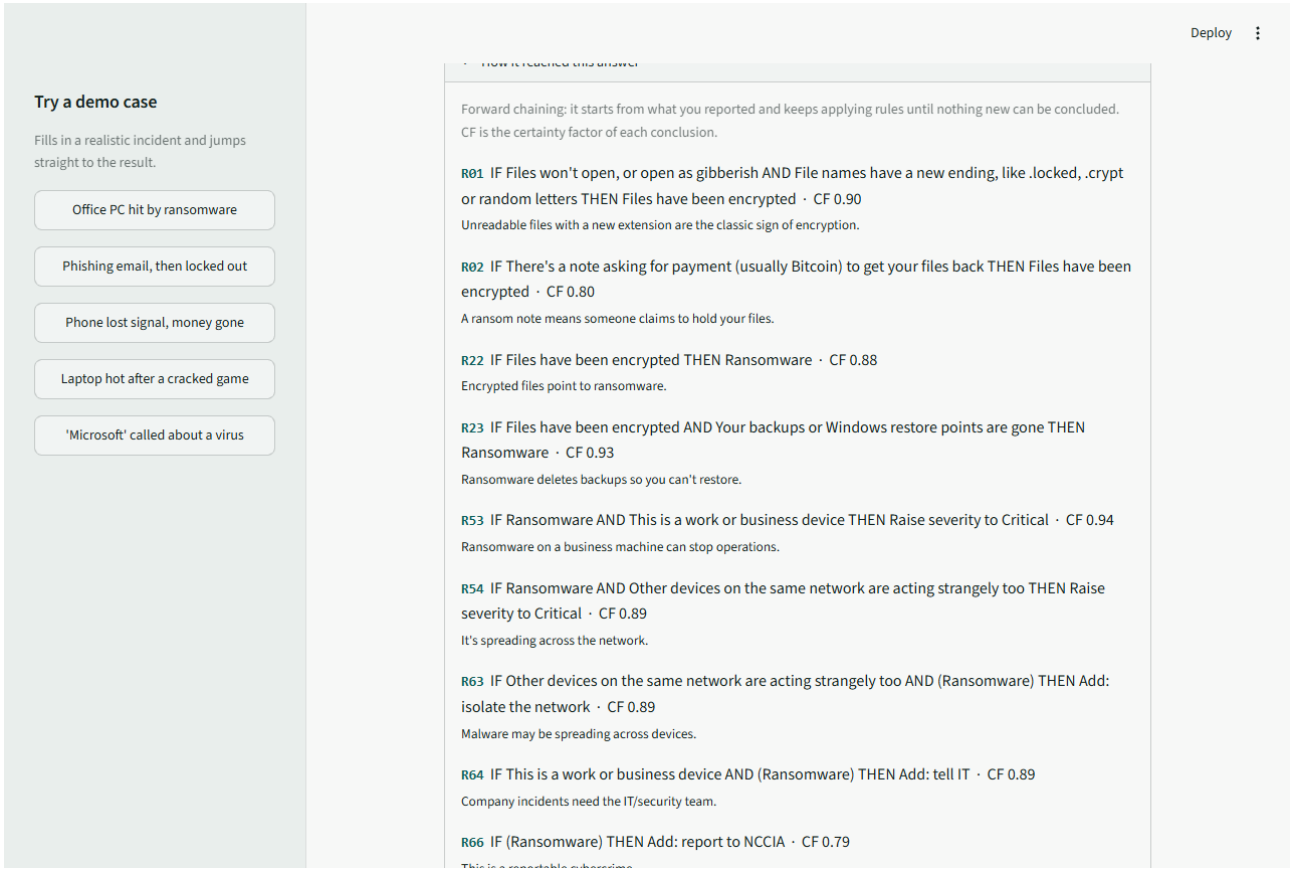



*Figure 4. The reasoning trace: each rule that fired, in order, with its CF.*

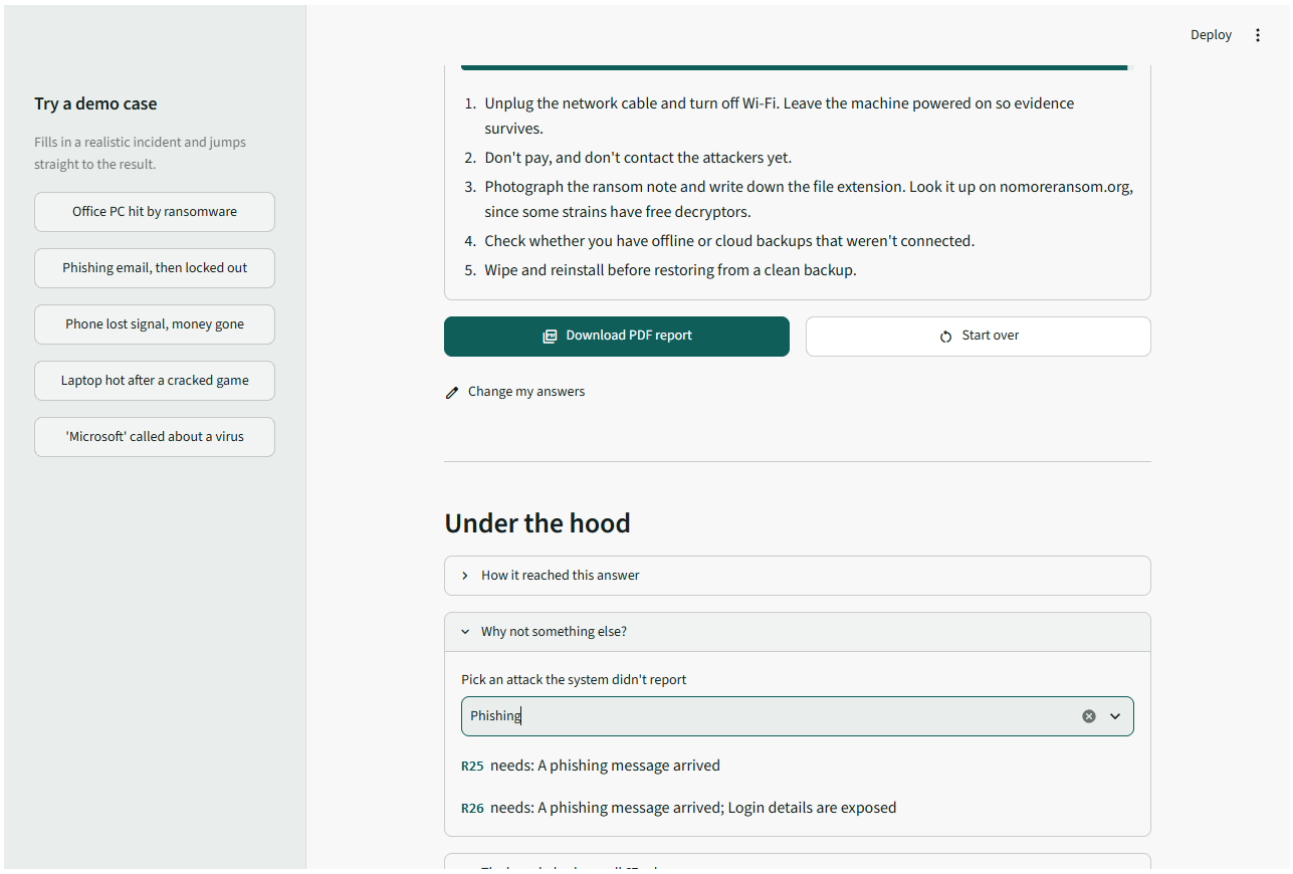



*Figure 5. 'Why not something else?': Phishing wasn't reported because no phishing message was marked.*

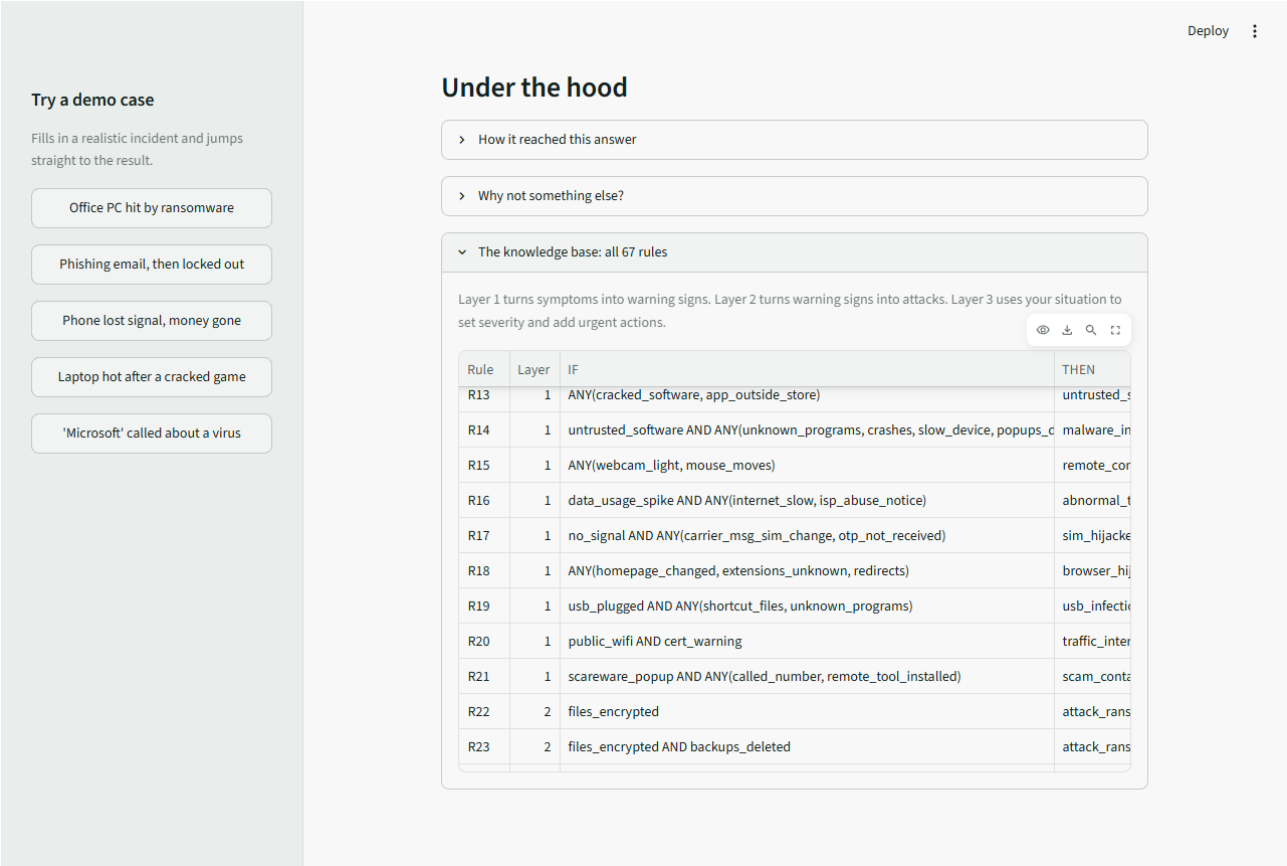



*Figure 6. The full knowledge base, viewable inside the app.*

## 6.3 The PDF incident report

The Download PDF report button builds a report with the fpdf2 library. It contains the severity, the urgent actions, every likely attack with its steps, what the user reported, and the rules that fired. The idea is that someone could hand it to their IT person, their bank, or attach it to a cybercrime complaint, instead of trying to remember what they saw.

## 6.4 Design choices

I kept the look plain on purpose: one dark teal accent, and four severity colours (green, amber, orange, deep red) that only appear where severity is shown. Someone using this is probably already stressed, so the most important thing, how bad it is, is the first thing on the result page. Every question is written the way a normal person would describe the problem, not in security jargon.

# 7. Testing

test_engine.py uses plain assert statements, no testing framework. It checks that:

- the MYCIN combination gives 0.8 for 0.6 and 0.5
- chaining works across two rules, with AND taking the minimum and the rule CF multiplying
- an OR rule doesn't fire when none of its options are true
- why_not() reports the right missing condition
- there are at least 50 rules, no duplicate rule IDs, and no rule refers to a fact that doesn't exist
- every attack has at least one rule that can conclude it
- each of the five demo cases puts the expected attack at the top

```
$ python test_engine.py
ok  Office PC hit by ransomware            -> Ransomware (0.99), Critical, 4 actions
ok  Phishing email, then locked out        -> Account takeover (0.96), High, 3 actions
ok  Phone lost signal, money gone          -> SIM swap (0.97), Critical, 2 actions
ok  Laptop hot after a cracked game        -> Cryptojacking (0.86), High, 1 actions
ok  'Microsoft' called about a virus       -> Tech-support scam (0.92), High, 2 actions
all checks passed: 67 rules, 51 questions
```

The fact-reference check matters more than it looks. A misspelled fact name in a rule wouldn't crash anything; the rule would just never fire, and nobody would notice. This check catches that.

# 8. Running and deploying it

To run it locally:

```
pip install -r requirements.txt
python test_engine.py
streamlit run app.py
```

For the live version, the code is on GitHub and deployed with Streamlit Community Cloud, which installs requirements.txt and runs app.py. Every push to the main branch redeploys automatically. The live link is https://cybertriage.streamlit.app/.

# 9. Limitations

The certainty factors on the rules are my own estimates from reading about these attacks, not numbers measured from real incident data, so the percentages show relative strength rather than true probability. The system also only knows the 15 attacks in its knowledge base. Something new, or two attacks happening together in an unusual way, can be missed or mixed up. And it can't look at the device itself; it only knows what the user tells it. It's a first-response guide, and for anything serious it tells people to get a professional involved.

# Appendix: all 67 rules

ANY(...) means at least one of those facts must be true. CF is the rule's certainty factor.

| Rule | IF | THEN | CF |
|---|---|---|---|
| R01 | files_unreadable AND odd_extensions | files_encrypted | 0.90 |
| R02 | ransom_note | files_encrypted | 0.80 |
| R03 | cpu_high_idle AND ANY(fan_loud, battery_drain, slow_device) | hidden_resource_use | 0.80 |
| R04 | clicked_link AND entered_password | credentials_exposed | 0.90 |
| R05 | breach_notice AND reused_password | credentials_exposed | 0.70 |
| R06 | suspicious_call | credentials_exposed | 0.50 |
| R07 | email_urgent AND email_sender_odd | phishing_message | 0.85 |
| R08 | email_urgent AND ANY(clicked_link, attachment_macro) | phishing_message | 0.60 |
| R09 | login_alert_unknown AND ANY(password_reset_unrequested, recovery_changed) | suspicious_logins | 0.80 |
| R10 | password_not_working AND ANY(recovery_changed, login_alert_unknown) | account_hijacked | 0.85 |
| R11 | unknown_programs AND antivirus_disabled | malware_installed | 0.85 |
| R12 | attachment_macro AND clicked_link | malware_installed | 0.70 |
| R13 | ANY(cracked_software, app_outside_store) | untrusted_software | 0.80 |
| R14 | untrusted_software AND ANY(unknown_programs, crashes, slow_device, popups_desktop) | malware_installed | 0.70 |
| R15 | ANY(webcam_light, mouse_moves) | remote_control_signs | 0.75 |
| R16 | data_usage_spike AND ANY(internet_slow, isp_abuse_notice) | abnormal_traffic | 0.80 |
| R17 | no_signal AND ANY(carrier_msg_sim_change, otp_not_received) | sim_hijacked | 0.90 |
| R18 | ANY(homepage_changed, extensions_unknown, redirects) | browser_hijacked | 0.75 |
| R19 | usb_plugged AND ANY(shortcut_files, unknown_programs) | usb_infection | 0.80 |
| R20 | public_wifi AND cert_warning | traffic_intercepted | 0.80 |
| R21 | scareware_popup AND ANY(called_number, remote_tool_installed) | scam_contact | 0.90 |
| R22 | files_encrypted | attack_ransomware | 0.90 |
| R23 | files_encrypted AND backups_deleted | attack_ransomware | 0.95 |
| R24 | files_missing AND ransom_note | attack_ransomware | 0.70 |
| R25 | phishing_message | attack_phishing | 0.80 |
| R26 | phishing_message AND credentials_exposed | attack_phishing | 0.90 |
| R27 | credentials_exposed AND ANY(suspicious_logins, account_hijacked, strange_transactions) | attack_account_takeover | 0.90 |
| R28 | account_hijacked | attack_account_takeover | 0.80 |
| R29 | suspicious_logins | attack_account_takeover | 0.60 |
| R30 | malware_installed AND remote_control_signs | attack_spyware | 0.85 |
| R31 | malware_installed AND ANY(suspicious_logins, strange_transactions, password_not_working) | attack_spyware | 0.65 |
| R32 | hidden_resource_use | attack_cryptojacking | 0.70 |
| R33 | hidden_resource_use AND untrusted_software | attack_cryptojacking | 0.85 |
| R34 | abnormal_traffic AND malware_installed | attack_botnet | 0.85 |
| R35 | isp_abuse_notice | attack_botnet | 0.75 |
| R36 | popups_desktop | attack_adware | 0.60 |
| R37 | browser_hijacked | attack_adware | 0.75 |
| R38 | untrusted_software AND malware_installed | attack_trojan | 0.85 |
| R39 | app_outside_store AND app_permissions | attack_trojan | 0.85 |
| R40 | sim_hijacked | attack_sim_swap | 0.90 |
| R41 | sim_hijacked AND strange_transactions | attack_sim_swap | 0.95 |
| R42 | suspicious_call AND otp_not_received | attack_sim_swap | 0.60 |
| R43 | posts_not_yours AND account_hijacked | attack_social_hijack | 0.90 |
| R44 | contacts_got_spam AND ANY(posts_not_yours, suspicious_logins) | attack_social_hijack | 0.80 |
| R45 | scam_contact | attack_tech_support_scam | 0.90 |
| R46 | scareware_popup | attack_tech_support_scam | 0.60 |
| R47 | traffic_intercepted | attack_mitm | 0.80 |
| R48 | router_settings_changed AND ANY(redirects, cert_warning) | attack_mitm | 0.80 |
| R49 | usb_infection | attack_usb_malware | 0.85 |
| R50 | breach_notice | attack_data_breach | 0.80 |
| R51 | boss_payment_request | attack_bec | 0.70 |

| Rule | IF | THEN | CF |
|---|---|---|---|
| R52 | boss_payment_request AND email_sender_odd | attack_bec | 0.90 |
| R53 | attack_ransomware AND ctx_business | sev_critical | 0.95 |
| R54 | attack_ransomware AND ctx_spread | sev_critical | 0.90 |
| R55 | attack_bec AND ctx_banking | sev_critical | 0.90 |
| R56 | attack_sim_swap AND ctx_banking | sev_critical | 0.90 |
| R57 | attack_account_takeover AND strange_transactions | sev_critical | 0.90 |
| R58 | attack_spyware AND ctx_sensitive | sev_high | 0.85 |
| R59 | attack_botnet AND ctx_business | sev_high | 0.80 |
| R60 | attack_data_breach AND reused_password | sev_high | 0.80 |
| R61 | attack_tech_support_scam AND remote_tool_installed | sev_high | 0.85 |
| R62 | ctx_banking AND ANY(attack_phishing, attack_account_takeover, attack_sim_swap, attack_bec, attack_tech_support_scam, attack_spyware, attack_trojan) | act_call_bank | 0.95 |
| R63 | ctx_spread AND ANY(attack_ransomware, attack_spyware, attack_cryptojacking, attack_botnet, attack_trojan, attack_usb_malware) | act_isolate_network | 0.90 |
| R64 | ctx_business AND ANY(attack_ransomware, attack_spyware, attack_cryptojacking, attack_botnet, attack_trojan, attack_usb_malware, attack_phishing, attack_account_takeover, attack_sim_swap, attack_bec, attack_tech_support_scam, attack_mitm, attack_data_breach) | act_inform_it | 0.90 |
| R65 | ANY(attack_account_takeover, attack_phishing, attack_data_breach, attack_spyware, attack_social_hijack, attack_mitm) | act_change_passwords | 0.90 |
| R66 | ANY(attack_ransomware, attack_bec, attack_sim_swap, attack_account_takeover, attack_tech_support_scam) | act_report_nccia | 0.80 |
| R67 | ANY(attack_ransomware, attack_spyware, attack_cryptojacking, attack_botnet, attack_trojan, attack_usb_malware) | act_disconnect | 0.90 |